WORK LOG

소상공인 AI 홍보 자동화 서비스 업무일지

소상공인이 간단한 브리프만 입력해도 Instagram, Threads, Blog용 홍보 콘텐츠와 배너 이미지를 생성하고, 결과 화면에서 수정 후 게시 준비까지 이어갈 수 있는 서비스형 MVP를 구현한 과정을 정리한 문서이다.

프로젝트명	정리 기간	개발 형태
소상공인 AI 홍보 자동화 서비스	2026-04-13 ~ 2026-04-27	FastAPI 기반 MVP 구현

1. 작업 기간과 진행 방식

이번 프로젝트는 크게 두 단계로 나누어 진행했다.

1단계: 2026-04-13 ~ 2026-04-22
문제 정의, 기능 기획, 채널별 요구사항 조사, 서비스 구조 구상

2단계: 2026-04-23 ~ 2026-04-27
실제 구현, 테스트, UI 개선, 문서화, 제출 자료 정리

2. 전체 작업 비중 요약

20%	20%	20%	20%	20%
서비스 구조 설계 및 FastAPI MVP 구성	AI 생성 구조 및 결과 스키마 설계	이미지 생성 및 배너 미리보기 개선	게시, 예약, 채널 연동 기능 구현	테스트, 문서화, 발표 및 제출 자료 정리

3. 일자별 업무일지

2026-04-13 ~ 2026-04-15

프로젝트 방향 확정

- 프로젝트 주제를 소상공인 대상 AI 홍보 자동화로 확정했다.
- 단순 문구 생성기보다 실제 게시 준비까지 이어지는 서비스 흐름을 목표로 잡았다.
- Instagram, Threads, Blog를 한 번에 다루는 구조가 더 설득력 있다고 판단했다.

집중한 내용

- 문제 정의
- 사용자 시나리오 구상
- 채널 선정 이유 정리

정리한 결론

사용자는 "글을 잘 쓰는 사람"이 아니라 "홍보할 시간이 부족한 사장님"이라고 보고, 만들기 쉬운 흐름을 우선했다.

2026-04-16 ~ 2026-04-18

서비스 구조와 기술 스택 구상

- `브리프 입력 -> 초안 생성 -> 이미지 수정 -> 채널별 검수 -> 예약/게시` 구조를 구상했다.
- FastAPI, Jinja2 SSR, JSON API를 함께 쓰는 방향을 검토했다.
- 향후 자동화 확장을 고려해 n8n 연동 가능성도 같이 정리했다.

집중한 내용

- 아키텍처 구상
- 기술 스택 선택
- 설명 가능한 구조 만들기

배운 점

프로젝트를 빨리 만드는 것과, 나중에 설명 가능한 구조로 만드는 것은 다르다는 점을 체감했다.

2026-04-19 ~ 2026-04-22

채널별 결과 구조와 MVP 범위 정리

- Instagram은 짧고 정보가 바로 보이는 문구, Threads는 대화형 문구, Blog는 검색형 제목 중심으로 기준을 세웠다.
- 이미지 생성은 provider 교체가 가능하도록, 게시는 채널별 adapter 구조로 가는 방향을 정했다.
- 로컬에서 빠르게 검증 가능한 MVP 범위를 우선 정리했다.

집중한 내용

- 결과 스키마 기획
- 이미지/게시 구조 분리
- 우선순위 조정

정리한 결론

모든 기능을 한 번에 완성하기보다, 먼저 실제로 작동하는 end-to-end 흐름을 만드는 것이 더 중요했다.

2026-04-23

FastAPI 기반 MVP 뼈대 구성

- 프로젝트 폴더 구조와 Python 가상환경, 실행 구조를 정리했다.
- `app`, `routes`, `services`, `schemas`, `templates`, `static`, `tests` 구조로 시작했다.
- 입력 페이지, 결과 페이지, health check 라우트를 포함한 MVP 기본 흐름을 완성했다.
- 제출용 보고서 PDF와 README 링크도 함께 정리했다.

당일 작업 비중

구조 설계 및 초기 세팅 45% / 기본 라우트·템플릿 구성 35% / 제출 자료 정리 20%

2026-04-24

이미지 생성 구조와 SD3.5 wrapper 연동

- 이미지 생성 구조를 provider 교체형으로 정리했다.
- 로컬 SD3.5 wrapper 연결 구조를 추가했다.
- wrapper 서버가 없거나 추론이 불안정할 때를 대비해 fallback 흐름을 넣었다.
- 배경 이미지와 배너 생성이 연결되는 전체 파이프라인을 정리했다.

해결 포인트
실제 모델이 불안정해도 발표나 시연에서 흐름이 끊기지 않도록 Pillow 기반 local fallback을 병행했다.

2026-04-25

AI 생성 구조와 저장 흐름 보강

- OpenAI 기반 광고 문구 생성 구조를 다듬고 mock fallback을 보완했다.
- Instagram, Threads, Blog용 채널 패키지 구조를 정리했다.
- 결과 저장소를 붙여 생성 이력, 재생성, 상태 관리를 할 수 있게 했다.
- 테스트 코드를 늘려 회귀를 막는 방향으로 작업했다.

배운 점
생성 품질만큼이나 생성 이후 결과를 저장하고 다시 꺼내 쓰는 구조가 서비스성에 큰 영향을 준다는 점을 느꼈다.

2026-04-26

결과 페이지 UX와 이미지 재생성 흐름 개선

- 브리프 입력 단계와 이미지 생성 단계를 분리했다.
- 결과 페이지 상단에서 참고 이미지를 넣고 다시 생성할 수 있도록 바꿨다.
- 생성 중 로딩 화면, 경과 시간, 현재 단계 표시를 추가했다.
- 3채널 미리보기와 채널별 문구 수정 UI를 정리했다.
- 업로드 이미지가 있을 때는 사진을 최대한 살린 배너 미리보기가 나오도록 보완했다.

해결 포인트
기능이 많아질수록 화면이 복잡해지는 문제를 탭, 카드, 작업 순서 분리로 해결했다.

2026-04-27

게시 연동, 네이버 블로그 UX, 발표 및 제출 자료 마무리

- Instagram, Threads direct API 게시 구조를 정리했다.
- Naver Blog는 Playwright 기반 연결·게시 흐름을 보완했다.
- 네이버 로그인 후 글쓰기 화면 진입을 감지하는 로직을 개선했다.
- 자동 감지가 실패할 때를 대비해 수동 완료 버튼도 추가했다.
- 브라우저 프로필 재사용으로 자동 로그인 흐름을 최대한 살리도록 보완했다.
- 발표 자료, README, API 가이드, 제출 문서를 최신 상태로 정리했다.

당일 작업 비중
게시 연동 및 네이버 블로그 UX 보완 45% / 레이아웃 마무리 25% / 발표·제출 자료 정리 30%

4. 프로젝트 전반에서 신경 쓴 부분

- 단순 생성이 아니라 이미지 재생성, 채널별 미리보기, 문구 수정, 계정 연결, 예약/게시까지 이어지는 흐름을 만들려고 했다.
- OpenAI 실패 시 mock fallback, SD 실패 시 local fallback, 네이버 블로그 연결 실패 시 수동 완료 fallback처럼 전체 흐름이 끊기지 않도록 구성했다.

- README, API 문서, 자동화 문서, 포트폴리오 타임라인, 발표 자료를 함께 정리해 설명 가능한 프로젝트 형태로 마무리했다.

5. 사용 기술 정리

Backend / Frontend

- Python 3.12
- FastAPI
- Uvicorn
- Pydantic / Pydantic Settings
- Jinja2 SSR
- HTML / CSS / Vanilla JavaScript

AI / Automation / Test

- OpenAI Responses API
- Pillow
- Local SD3.5 wrapper 구조
- Playwright
- n8n 연동 구조
- pytest / pytest-asyncio

6. 최종 소감

이번 작업은 단순히 AI API를 붙이는 수준이 아니라, 실제로 사용할 수 있는 서비스 흐름을 설계하는 경험에 가까웠다. 기능 하나를 구현할 때마다 "이 결과를 사용자가 다음에 어떻게 쓸까"를 함께 고민해야 했고, 그 과정에서 생성, 저장, 수정, 게시 준비, 문서화까지 전체 서비스를 보는 시야가 중요하다는 점을 배울 수 있었다.

특히 fallback 구조, 결과 페이지 UX, 채널별 게시 전략 분리, 제출 문서 정리를 통해 단순 데모를 넘어 설명 가능한 서비스형 MVP로 프로젝트를 마무리할 수 있었다.